Where the round timings come from, what weak hardware does to them, and the share-lock with every constant defined

Thanks for the three confirmations: Option A stands, the seed comes from the buried anchor (1B), and certificate gossip (3A) is a go. That unblocks the implementation. This letter answers the two questions you attached to the remaining decisions: how the round-timing constants are derived and what happens on poor hardware (your condition for the committee size), and a from-zero explanation of the share-lock with every constant defined (your condition for 3B and the 3A residual). The timing numbers below are MEASURED, not estimated: I benchmarked the exact signature-verification code the nodes run (the benchmark is checked in as `bls-share-verify-bench.cpp` next to this note, with build instructions, so anyone can reproduce it on any machine).

1. Where 500, 700, 25 and 35 actually come from

Honest answer first: they are tuned engineering constants, not derived from a formula. The history is in the code comment (pos_producer.cpp:952-960): the original values were a flat 500 ms collection window and 700 ms rounds, sized for the early small committees. When the committee went to 100, those stalled it, because at large K two things must fit inside the window and round that do not exist at K=5: every member's share must cross a multi-hop gossip mesh, and every node must verify up to K incoming shares. So a per-member slope was added, 25 ms of window and 35 ms of round per member, chosen so that a 100-member committee gets about 3 seconds to collect, which the live network then validated (the current chain has certified its whole history inside these timings).

So the structure is: the flat 500/700 ms covers what does not grow with K (one proposal crossing the network and being validated), and the per-member slope covers what does (share traffic and share verification). Your question is really about the slope: what is the true per-member cost, and how badly does it vary with hardware?

2. The measured per-member cost, on good and bad hardware

There are only two per-member costs. The network one is small and does not vary much: a share is ~300 bytes, so even 250 of them are ~75 KB per block, and their propagation is bounded by mesh latency (a few hundred ms intercontinental), not by K. The one that varies with hardware is cryptographic: today every node verifies every incoming share individually, two BLS pairing checks per share (a proof-of-possession and the signature itself, OnShare, pos_producer.cpp:1227-1228). I measured that exact code path (same library, same ciphersuite, same calls), on my ordinary dev laptop at full speed, and then again with the process capped at one quarter of one CPU core, which is a fair proxy for a cheap single-board machine or a starved VPS:

what a node must do per block	dev laptop	quarter of one core
verify ONE share (today's path: PoP + signature)	2.3 ms	8.6 ms
verify 100 shares one by one (K=100, today)	0.23 s	0.86 s
verify 250 shares one by one (K=250, today)	0.57 s	2.1 s
verify 250 shares BATCHED (one aggregate check)	0.035 s	0.14 s

Two readings:

- **Your instinct is right about today's code.** With per-share verification, K=250 on weak hardware eats ~2.1 s of the 6.75 s window. Add slow I/O and a congested link and a bad machine could plausibly need several seconds, which is exactly the "much more than optimal" you feared.
- **The cost all but disappears under the design you already approved.** Every share signs the SAME block hash (that is the whole point of keeping the certificate out of the signed hash), and BLS lets you verify any number of same-message signatures with ONE pairing check after cheap aggregation. The function already exists in our tree (`BlsFastAggregateVerify`, bls.cpp:101); it is used for the assembled certificate but not yet for incoming shares. And the per-share proof-of-possession check vanishes entirely under Option A, because possession is proven once at registration instead of re-verified in every block. Batched, the entire K=250 verification burden is 35 ms on a laptop and 139 ms on a quarter of a core. That is noise against a 6.75 s window.

So the implementation plan (part of the Option A build, not an extra project) is: shares are batch-verified, possession moves to registration, and the per-member slope then covers only network traffic. After that change, K=250 fits comfortably inside the SAME time profile that K=100 uses today, and the current formula (6.75 s window, 9.45 s rounds at 250) keeps roughly 3x headroom on top as a safety margin.

3. Why a wrong timing can cost seconds but not the chain

This is the property that makes the constants safe to be imperfect, and it also connects your two questions.

First: the window and round lengths are LOCAL LIVENESS timings, not consensus rules (pos_producer.cpp:957-958 says exactly this). No block is valid or invalid because of them. A node that is too slow for the current round simply contributes its share late or misses that round; nothing it produces is rejected, and nothing it accepts is wrong. I will also make them configurable per node (with the formula as default), so an operator on weak hardware can lengthen them without asking anyone.

Second, and more important: certification pace is set by the FASTEST quorum, not the slowest member. Any node that has collected a quorum of shares assembles the certificate

(pos_producer.cpp:1054-1057), so the block certifies as soon as the quickest 126 of 250 have signed and their shares have reached any single node. The slowest half of the committee, the cheap laptops and the far-away nodes, contribute nothing to latency: their shares are simply not needed. A geographically spread committee therefore does not certify at the speed of its worst region; it certifies at the speed of its best majority.

Third: the one place where slow propagation could historically do real damage is the round-boundary re-vote, your 3B, where a certificate that completes near the boundary races the clock. That is a SAFETY coupling of the timing constants, and it is exactly what the share-lock removes. With the share-lock in place, every timing mistake in the system, too tight a window, a slow continent, an overloaded node, degrades into seconds of delay at worst. Without it, one unlucky race can split the chain. That is the strongest reason to fold the share-lock into the same batch: it converts the whole timing topic from a safety question into a tuning question.

On block frequency: no change needed or recommended. The 30-second spacing is enforced by the slot rule independently of these timings; the window and rounds run inside that interval. The only thing that grows with K is the failover granularity when a leader is dead (each skipped leader costs one round, 9.45 s at K=250 against 4.2 s at K=100). Blocks led by a live leader, which is nearly all of them, are unaffected.

And to close the loop on committee size: with batching implemented, nothing in the latency picture argues against 250. But you do not have to take the argument on faith, because the constant does not need to be locked until re-genesis: the sandbox in the test plan will run K=250 with throttled-CPU nodes and injected latency and measure round convergence directly. My proposal: build against 250, measure it in the sandbox, and lock the value on the measurement. If the sandbox says otherwise, dropping to 200 or 128 is a one-line change.

4. The share-lock, from zero

Every constant first. All of these exist today except the last two, which are the fix.

name	value	what it is
slot interval	30 s	nominal block spacing (consensus: a block's time gate)
T	per height	the round-0 leader's block timestamp, read off its proposal
collection window	500 ms + 25 ms x K (3.0 s at K=100, 6.75 s at 250)	after T, nodes gather competing proposals, then all back the best leader
round	700 ms + 35 ms x K (4.2 s at K=100, 9.45 s at 250)	the time ONE leader gets to be certified before the committee moves to the next leader at the SAME height

name	value	what it is
grace (new)	one round	extra wait before a signer of X may back a rival at X's height
escape gap (exists)	3 BITCOIN blocks, ~30 min	whitepaper 3.8 anti-freeze valve: after it, a below-quorum block is accepted so the chain can never freeze

One correction to your restatement, because it matters for trust in the mechanism: there is no locally-calculated period in which signers are "expected to receive the block." Every node derives the current round from the SAME anchor, the timestamp T written inside the round-0 leader's proposal, so all honest nodes are always in the same round without synchronized clocks (pos_producer.cpp:942-951). The "about 8 seconds" you remember is T + 3.0 s (window) + 4.2 s (one round) = T + 7.2 s, the end of round 0 at K=100. It is not measured or averaged from anything; both numbers come from the two formulas above with K = 100.

The failure, in those terms. Block X gathers its 51st share at T + 7.0 s, just inside round 0. Some node assembles the certificate and starts flooding it, but flooding a block takes a second or two, and at T + 7.2 s the clock rolls into round 1. Every member that has not yet seen assembled-X now signs round 1's leader at the same height. Their round-0 shares for X are already out and remain valid, so X's certificate still completes. If the round-1 block also reaches 51, there are two certified blocks at one height: a permanent split, made entirely of honest nodes following the clock.

The rule. A member that has signed block X at height H does not sign ANY other block at H until X can no longer be certified. Since "can no longer" is not directly observable, the release event is all three of:

1. the round in which the member signed X has ended (the clock passed its boundary);
2. the member has ASKED: it sends a small query to its peers, "does anyone hold a certificate for height H?", and waits for the answers. Certificates are now tiny self-verifying objects that travel on their own (the 3A fix you confirmed), so if X certified anywhere, the ~300-byte proof of that is one round-trip away;
3. one grace round has passed with no certificate appearing.

Why the grace is "one round" and what it means: the round length is already the system's own budget for "one certification attempt completes and propagates." The grace reuses that same budget once more, for the specific block the member personally signed. It is not an empirical average, and it does not need to be accurate: if it is too short for some strange network moment, the certificate query (step 2) is still there, and the escape gap (below) still backstops everything. It also scales automatically: retune the round formula and the grace follows.

What "waiting" looks like, and who pays. Only the members that signed X hold back, and only at height H. Everyone else signs the round-1 leader immediately. This gives the rule its teeth through plain arithmetic: for X's certificate to be a live threat, at least a quorum Q must have signed X; those are all locked; at most K - Q members remain free, and K - Q < Q under your cap; so no rival can reach

quorum while the lock holds, and the two-certificates situation is impossible rather than unlikely. Conversely, if too few members signed X for it to ever certify, then fewer than Q are locked, and the round-1 leader can certify without waiting at all. The grace therefore delays the chain ONLY when X was genuinely on the verge of certifying, which is precisely the moment you want everyone to pause for a few seconds. The steady-state cost is zero; the cost when a leader dies mid-certification is one round.

Where the escape gap enters (the 3A residual). Rounds and the grace handle races measured in seconds. A network PARTITION can last longer than any number of rounds, and a member cut off from the side where X certified can query its peers forever and hear nothing: its peers are all on the wrong side. For that case the lock keys on the escape gap instead of the clock: at a height where the member signed a share, it does not join a chain that excludes X until the Bitcoin chain has advanced 3 blocks past the parent's anchor (the same ~30-minute valve the whitepaper already uses to relax the quorum during a stall) AND its certificate query has come back empty. The worst case then degrades from "two finalized histories" to "the cut-off minority waits out the partition," which is the trade Sequentia makes everywhere else, and the majority side never waits at all.

To your specific question "they wait for what, for other leaders to try certifying a second block?": no. During the grace the locked members are not waiting for anyone to certify anything; they are waiting for EVIDENCE about the block they already signed, either its certificate arriving (then they adopt X and no rival ever existed) or silence past the grace (then X is dead and they are free to back the next leader). The next leader's attempt proceeds in parallel among the unlocked members; it simply cannot reach quorum until the locked members are released, which is the whole point.

5. Where this leaves the decisions

- Option A: confirmed. Seed 1B (buried anchor): confirmed. Certificate gossip (3A): confirmed. Implementation of all three is unblocked and will proceed tests-first.
- Committee size: my recommendation stays 250, now with the measured basis above (batched verification makes K=250 cheaper on weak hardware than K=100 is today without it), and with the sandbox measuring round convergence at K=250 on throttled hardware BEFORE the value locks at re-genesis. Block frequency stays 30 s in any case.
- Share-lock (3B + the 3A partition residual): the mechanism is exactly section 4. Given that it also removes the only safety coupling of the timing constants you were worried about, I recommend it goes in the same batch. If section 4 reads right to you, that is the last go/no-go on the list.

Benchmark source: doc/sequentia/bls-share-verify-bench.cpp (build lines in the header; mirrors src/bls.cpp call for call). Timing formulas: pos_producer.cpp:961-962. Nothing is in consensus code yet.